

JavaScript — Execution Context & Call Stack

1. What is Execution Context?

Execution Context is the environment in which JavaScript code is executed.

In simple words:

JavaScript creates an environment to keep track of variables, functions, scope, `this`, and the code that needs to run.

Whenever JavaScript runs a program, it creates an execution context.

2. Types of Execution Context

JavaScript has three commonly discussed types:

1. Global Execution Context

Created when JavaScript starts executing a script.

It handles the code that is not inside a function.

```
Program starts
  ↓
Global Execution Context
```

There is normally one global execution context for a script.

2. Function Execution Context

Created every time a function is called.

```
function greet() {
  console.log("Hello");
}

greet();
```

When `greet()` is called, JavaScript creates a new Function Execution Context for it.

```
Global Context
  ↓
greet() Context
```

When the function finishes, its context is removed from the Call Stack.

3. Eval Execution Context

Created when JavaScript executes code using `eval()`.

```
eval("console.log('Hello')");
```

It is rarely used in modern development and is generally avoided because of security and performance concerns.

For interviews, know that it exists, but **Global and Function Execution Contexts are much more important.**

3. How JavaScript Executes Code

A simplified execution process is:

```
JavaScript Code
  ↓
Execution Context Created
  ↓
Creation Phase
  ↓
Execution Phase
  ↓
Code Executes
```

For the global code, JavaScript first creates the Global Execution Context.

When a function is called, another Function Execution Context is created.

4. Creation Phase

Before actually executing the code, JavaScript prepares the execution environment.

During this stage, JavaScript sets up things such as:

- Variables/bindings
- Function declarations
- Scope information
- `this`
- Access to outer environments

Example:

```
var x = 10;

function test() {
  console.log("Hello");
}
```

Conceptually, before execution:

```
x      → undefined
test   → function
```

The function declaration is already available.

This preparation is one of the reasons we can observe **hoisting**.

5. Execution Phase

After the environment has been prepared, JavaScript executes the code.

For:

```
var x = 10;
console.log(x);
```

Conceptually:

Creation Phase:

`x → undefined`

Execution Phase:

`x → 10`

`console.log(x)`

Output:

10

So remember:

Creation Phase → Prepare

Execution Phase → Run

6. Hoisting and Execution Context

Hoisting is closely related to the creation/setup phase.

`var`

```
console.log(x);  
var x = 10;
```

Conceptually:

```
var x;  
  
console.log(x);  
  
x = 10;
```

Therefore:

undefined

Function Declaration

Function declarations are available before their position in the source code:

```
greet();

function greet() {
  console.log("Hello");
}
```

The function can be called successfully.

`let` and `const`

`let` and `const` bindings are created during environment setup, but they cannot be accessed before initialization.

That period is called the:

Temporal Dead Zone (TDZ)

Accessing them before initialization results in a `ReferenceError`.

7. What Does an Execution Context Contain?

An execution context maintains information required to execute code.

Important parts include:

Variables and Function Bindings

JavaScript needs to know what variables and functions are available.

Lexical Environment

It keeps track of identifiers and their relationship with the surrounding scope.

This is important for:

- Scope
- Nested functions
- Closures

`this` Binding

The value of `this` is determined according to the execution context and how the function is called.

For example, `this` behaves differently in:

- Global code
- Regular functions
- Object methods
- Arrow functions
- Classes

Outer Environment Reference

A function can access variables from its outer lexical environment.

Example:

```
const name = "Satyam";

function greet() {
  console.log(name);
}
```

`greet()` can access `name` because its environment has access to the outer environment.

This concept becomes very important when learning **closures**.

8. Function Execution Context

Every time a function is called, JavaScript creates a new execution context.

Example:

```
function add(a, b) {
  const result = a + b;
  return result;
}

add(10, 20);
```

When `add()` is called, JavaScript creates a new Function Execution Context containing information such as:

```
a      → 10
b      → 20
result → 30
this   → depends on how function is called
```

When the function returns, its execution is complete.

9. Each Function Call Gets Its Own Context

Consider:

```
function test() {
  const x = 10;
}

test();
test();
```

Each call creates a new Function Execution Context.

```
First test() call
    ↓
Context created
    ↓
Function finishes
    ↓
Context removed

Second test() call
    ↓
New context created
```

The two calls do not share the same local execution context.

10. What is the Call Stack?

The **Call Stack** is a data structure used by JavaScript to keep track of function calls.

It follows:

LIFO

Last In, First Out

Think about a stack of plates:

```
Plate 3 ← first removed  
Plate 2  
Plate 1
```

The last plate added is the first one removed.

The Call Stack works similarly.

11. How the Call Stack Works

Suppose:

```
function first() {  
  second();  
}  
  
function second() {  
  console.log("Hello");  
}  
  
first();
```

Initially:

```
Call Stack  
-----  
Global
```

When `first()` is called:

Call Stack

`first()`
Global

Then `second()` is called:

Call Stack

`second()`
`first()`
Global

`second()` is at the top, so it executes first.

After it finishes:

Call Stack

`first()`
Global

Then `first()` finishes:

Call Stack

Global

12. Execution Context + Call Stack Relationship

These concepts work together.

Whenever a function is called:

Function Called

↓

Function Execution Context Created

↓
Context pushed onto Call Stack
↓
Function executes
↓
Function finishes
↓
Context removed from Call Stack

This is the most important relationship to understand.

13. Example of Nested Calls

Consider:

```
function A() {  
  B();  
}  
  
function B() {  
  C();  
}  
  
function C() {  
  console.log("Hello");  
}  
  
A();
```

The Call Stack becomes:

```
C()  
B()  
A()  
Global
```

Because of LIFO, `C()` executes first.

After `C()` finishes:

```
B()  
A()  
Global
```

Then `B()` finishes:

```
A()  
Global
```

Finally:

```
Global
```

14. Stack Overflow

The Call Stack has a limited size.

If functions continuously call themselves without stopping, the stack keeps growing.

Example:

```
function test() {  
    test();  
}  
  
test();
```

This creates:

```
test()  
test()  
test()  
test()  
...
```

Eventually JavaScript throws:

```
RangeError: Maximum call stack size exceeded
```

This is called **Stack Overflow**.

15. Recursion and Call Stack

Recursion means a function calls itself.

Example:

```
function count(n) {  
  if (n === 0) return;  
  
  console.log(n);  
  count(n - 1);  
}  
  
count(3);
```

The stack grows:

```
count(3)  
count(2)  
count(1)  
count(0)
```

Then the functions return one by one:

```
count(0) → finished  
count(1) → finished  
count(2) → finished  
count(3) → finished
```

The **base condition** prevents infinite recursion.

16. Synchronous Code and Call Stack

Normal JavaScript code is executed synchronously.

Example:

```
console.log("A");  
console.log("B");  
console.log("C");
```

The code executes in order:

```
A  
↓  
B  
↓  
C
```

The Call Stack handles the currently executing JavaScript code.

17. What Happens with Asynchronous Code?

Consider:

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 1000);  
  
console.log("C");
```

Output:

```
A  
C  
B
```

Why?

The timer does not block the Call Stack for one second.

The runtime handles the timer separately. Once the timer is ready, its callback is scheduled to run when the Call Stack is available.

Simplified:

```
console.log("A")
  ↓
Call Stack

setTimeout()
  ↓
Timer handled by runtime

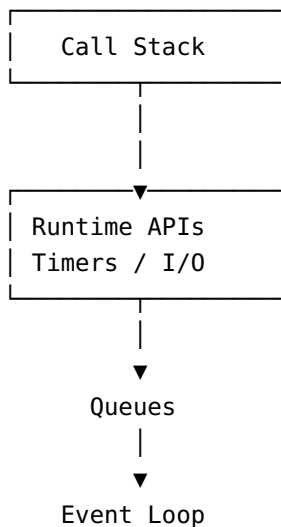
console.log("C")
  ↓
Call Stack continues

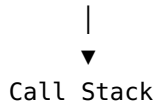
Timer becomes ready
  ↓
Callback waits for execution
  ↓
Call Stack
  ↓
console.log("B")
```

This is where the **Event Loop** becomes important.

18. Call Stack and Event Loop

For asynchronous JavaScript, you should understand these main components:





The exact queues and scheduling details depend on the runtime, but the basic idea is:

The Event Loop coordinates when queued asynchronous callbacks can get a chance to execute on the Call Stack.

19. `setTimeout(fn, 0)` Does Not Mean Immediate

Consider:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

Even though the delay is `0`, the callback does not interrupt the current synchronous code.

The current code must finish first.

So:

```
0ms
```

does **not** mean:

Execute immediately

It means the callback can become eligible without a deliberate time delay, but it still has to wait for the appropriate scheduling point.

20. Execution Context vs Call Stack

These are not the same thing.

Execution Context	Call Stack
Environment for executing code	Data structure tracking active calls
Contains execution-related information	Stores active execution contexts/call frames
Created when execution begins or a function is called	Context is pushed when function execution begins
Deals with variables, scope, <code>this</code> , etc.	Follows LIFO
Describes the execution environment	Tracks execution order

Easy way to remember:

Execution Context = Environment

Call Stack = Tracking system

21. Execution Context vs Scope

These are also different.

Execution Context

The complete environment needed to execute code.

Scope

Determines where a variable can be accessed.

Example:

```
const x = 10;

function test() {
  const y = 20;
}
```

`test()` has its own execution context.

Its scope determines which variables are accessible inside it.

So:

Execution Context → Execution environment

Scope → Variable accessibility

22. Important Execution Flow

When JavaScript starts:

```
JavaScript program
  ↓
Global Execution Context created
  ↓
Creation/setup phase
  ↓
Execution phase
  ↓
Code runs
```

When a function is called:

```
Function call
  ↓
Function Execution Context created
  ↓
Pushed onto Call Stack
  ↓
Function executes
  ↓
```

Function returns



Context removed from Call Stack

23. Complete Flow Example

```
const x = 10;

function one() {
  const y = 20;
  two();
}

function two() {
  console.log(x);
}

one();
```

Step 1 — Global Context

JavaScript creates the Global Execution Context.

It knows about:

```
x
one
two
```

Step 2 — `one()` is called

A Function Execution Context is created for `one()`.

```
Call Stack
├── one()
└── Global
```

Step 3 — `two()` is called

Another Function Execution Context is created.

Call Stack

two()
one()
Global

Step 4 — `two()` **finishes**

Call Stack

one()
Global

Step 5 — `one()` **finishes**

Call Stack

Global

The global code then continues/finishes.

24. Important Interview Points

Execution Context

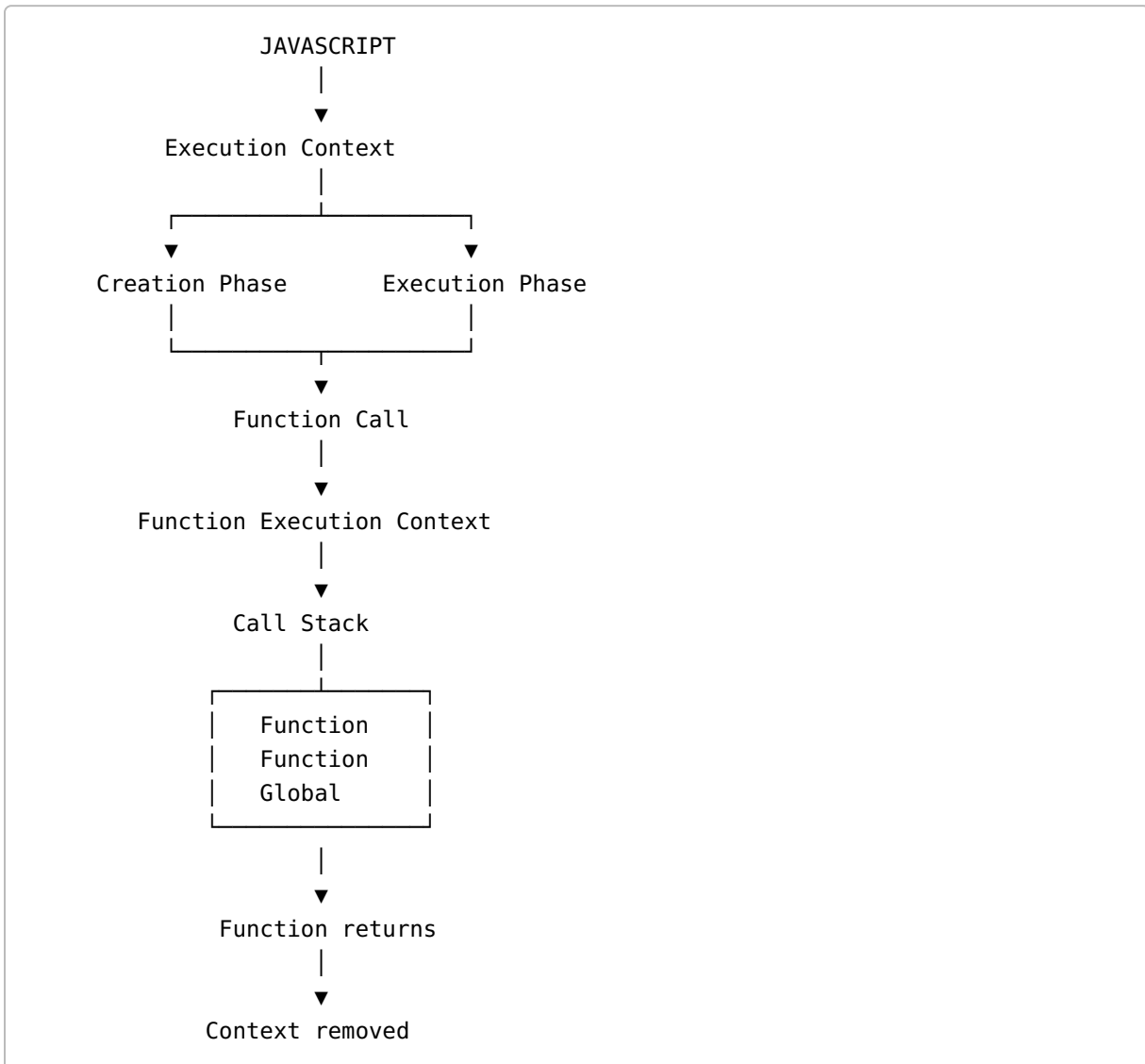
- It is the environment where JavaScript code executes.
- Global code gets a Global Execution Context.
- Every function call creates a Function Execution Context.
- Execution contexts contain information needed for execution.
- Creation/setup happens before actual execution.
- Hoisting is related to declaration setup.
- `let` and `const` have the Temporal Dead Zone before initialization.

Call Stack

- It tracks active function calls.
- It follows **LIFO**.
- A function call adds a context/frame to the stack.
- Returning from a function removes it.
- Recursive calls can cause stack overflow.
- The Call Stack handles synchronous JavaScript execution.
- Asynchronous callbacks execute when they are scheduled and the Call Stack is available.

25. Final Mental Model

Don't memorize too much. Just remember this flow:



One-line Definitions

Execution Context:

The environment in which JavaScript executes code.

Global Execution Context:

The initial execution context created for global code.

Function Execution Context:

A new execution context created whenever a function is called.

Call Stack:

A LIFO structure that keeps track of currently executing function calls.

LIFO:

Last In, First Out.

Stack Overflow:

Happens when the Call Stack exceeds its available limit, often because of infinite recursion.

Event Loop:

Coordinates when asynchronous callbacks can be executed after the Call Stack is available.

Quick Revision

```
Execution Context
  ↓
Environment for code execution

Types
  ↓
Global
Function
Eval

Execution
  ↓
Creation/Setup Phase
  ↓
Execution Phase

Function Call
  ↓
New Function Context
```

↓
Call Stack
↓
Execute
↓
Return
↓
Remove from Stack

Call Stack
↓
LIFO
↓
Last In → First Out

Too many calls
↓
Stack Overflow

Interview-ready answer

When JavaScript starts running a program, it creates a Global Execution Context. During execution, whenever a function is called, JavaScript creates a Function Execution Context for that call and puts it on the Call Stack. The Call Stack follows LIFO, so the most recently called function executes and returns first. After a function finishes, its context is removed from the stack.